Algorithm Notes

Let's start off with something a bit fun: showing that $O(log(n)) == O(log(n+1))$.

For $n > 1$, we have $n < n+1 < n^2$ and hence $log(n) < log(n+1) < log(n^2) = 2log(n)$. So, $log(n+1)$ is eventually (more precisely, for $n > 1$) bounded above and below by constant multiples of $log(n)$.

Binary Search Trees

Definition: A *binary search tree* is a binary tree where the key of any node n is $\geq$ all keys in n's left subtree and $\leq$ all keys in n's right subtree.

```
def InOrderTreeWalk(v):  # Print vertices in order
    InOrderTreeWalk(v.left)
    print(v.key)
    InOrderTreeWalk(v.right)
```

Linear Complexity of InOrderTreeWalk: Clearly, linear is the best we can hope for, since all vertices are visited. Suppose that InOrderTreeWalk takes time c on an empty subtree and takes time bounded by d to execute the part of itself which is not inside recursive calls. We prove by induction that, for a tree of size n, $T(n) \leq (c+d)n + c$. Well, this is clearly true for $n = 0$. Let us assume that our tree's left subtree has size k and its right subtree has size $n-k-1$; then

$$T(n) \leq T(k) + d + T(n-k-1) \leq (c+d)n + c$$

.

$O(height)$ Operations: Search, Min, Max, Successor, Predecessor

Search

```
def Search(k):  # Iterative Version
    x = root
    while x != NIL and x.key != k:
        if key < x.k:
            x = x.left
        else:
            x = x.right
    return x
```

There is also a recursive version.

Min

Just start at the root and keep going left until you can't any more

Max

Same as *Min* but keep going right

Observation

For a node x in a BST, each of $Predecessor(x)$ and $Successor(x)$ is either an ancestor or a descendant of x. Indeed, suppose that m is a node which is neither an ancestor nor a descendant of x and let a the the lowest common ancestor of x and m. Then x and m belong to opposite subtrees of a. If x is in the left subtree of a, then $x < a < m$, meaning that m is neither the successor nor the predecesor of x. If x is in the right subtree of a, then $m < a < x$, meaning that m is neither the successor nor the predecesor of x.

Successor

The Successor *Procedure:* If x has a right subtree, then $Successor(x)$ is just $Min(x.right)$. If x has no right subtree, then to find $Successor(x)$, go up the tree in a northwesterly direction for as far as possible; when you can no longer move in a northwesterly direction (e.g. because your current node is the left child of its parent), return the current node's parent.

Predecessor

Symmetric to *Successor*.

Insert

The Insert *Procedure:* Very simple. We are going to insert the node at the bottom of the tree, so we are looking for a leaf node to be the parent of the new node. We find this node by starting at the root and going down the tree in the most obvious way. Takes $O(height)$ time.

Remove

Suppose that we wish to remove a node z.

If z has no children, then we just remove z. If z has only one child, then we just replace z with its child. So, the only interesting case is where z has two children. We find z's successor y, which lies in z's right subtree and has no left child. There are two cases:

- y is z's right child; in this case, we just replace z by y.
- otherwise; replace y by its own right child, then replace z by y.

If z has no children or only one child, then removal takes constant time. If z has two children, then every operation in the removal procedure takes constant time except for finding z's successor, so that removal takes $O(\log(height(z)))$ time.

Building A Binary Search Tree

Theorem: Building a binary search tree by randomly ordering the keys and then inserting them one by one produces a tree with an expected height of $O(logn)$.

We won't prove this theorem but it's worth knowing. It is similar to quicksort, where the average case is much better than the worst case.

Red-Black Trees

The idea of a red-black tree (RBT) is to keep a BST "balanced" so that its height stays at $O(logn)$. More precisely: by constraining the node colors on any simple path from the root to a leaf, RBTs ensure that no such path is more than twice as long as any other, so that the tree is approximately balanced.

Note: When discussing RBTs, we shall consider all actual nodes to be "internal", by the convention that a node which has no left child (or right child or parent) has *NIL* as its left child (or right child or parent).

Definition: A binary tree is a RBT if

1. Every node is either red or black
2. The root is black
3. Every leaf (NIL) is black
4. If a node is red, then both its children are black
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes

Definition: The *black height*, $bh(x)$, of a node x is the number of black nodes (not including x itself) on any simple path from x down to a leaf.

Theorem: RBTs are balanced i.e. a RBT with n internal nodes has height at most $2log(n+1)$. **Proof:** For any node x, is easy to show by induction on the height of x that the subtree rooted at x contains at least $2^{bh(x)} - 1$ internal nodes. Now, let h be the height of the tree. By the RBT definition, at least half the nodes on any simple path from the root to a leaf, not including the root itself, must be black, so the black-height of the root is at least $h/2$, so that $n \geq 2^{h/2} - 1$ and hence $n + 1 \geq 2^{h/2}$. Taking logs yields the result.

RBT Operations

All Take $O(height)$ Time, Which Is $O(logn)$ Because RBTs Are Balanced

Search, Min, Max, Predecessor, Successor All are the same as for any BST.

Insert Let's assume we are inserting a node into a nonempty RBT (since the empty case is trivial). First, we insert the node as we normally would into

any BST, then we colour the new node red. Then we do some "fixups": if the new node's parent is black, then we are done; if the new node's parent is red, then we have a double red and we have work to do. The two major cases are distinguished by whether the new node's uncle is red or black.

- Red uncle: do some recolouring
- Black uncle: do some rotation and recolouring

Since there is a fixed upper bound on the number of operations required for a fixup (regardless of the number of elements in or the height of the RBT), insertion in an RBT has complexity the same as for any BST, namely $O(height)$.

For details, see RBT-Insert-GeeksForGeeks

Delete Deleting a node from an RBT is more complex than inserting a node. We won't treat it here.

Merge Sort

Complexity

Assume for convenience that there are $n = 2^m$ elements, so that there are m steps. Step i does 2^i split steps and the merging totals n steps. So that's $\sum_{i=0}^{m} 2^i + mn$ steps i.e. $2.2^m - 1 + mn$ steps i.e. (since $m = log(n)$) $O(nlog(n))$.

Heap Sort

Heaps

Definition: A *complete binary tree* is a binary tree which is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.

Suppose we list the nodes of a complete binary tree in an array A in the obvious way (with indices starting at zero for both the array and the tree levels). Let x be the nth element at level m. Then x has $2^0 + ... + 2^{m-1} = 2^m - 1$ elements above it and n elements to its left, so x is at position $2^m + n - 1$ in the array. Now, using this formula to express the positions of x's parents and children, we get the following formulae:

- `left_child(i) = 2i`
- `right_child(i) = 2i + 1`

from which it follows easily that

- `parent(i) == floor(i/2)`

Max Heaps (Min Heaps Are Obviously Similar)

Definition: A *max heap* is a complete binary tree where the key of any node is greater than that of its children.

Operations On Max Heaps

Max_Heapify

Suppose that A is a complete binary tree and the left and right subtrees of the root are max heaps. We can turn A into a max heap by applying the *Max_Heapify* procedure.

Definition: *Max_Heapify* allows the root key to "float down" the tree by swapping the key with the key of its smaller child until the key is greater than that of both its children.

Clearly, *Max_Heapify* turns the tree back into a heap and runs in $O(log(n))$ time.

Build_Max_Heap

Suppose that A is a complete binary tree (i.e. just an array) and we want to turn it into a max heap.

Definition: *Build_Max_Heap* starts at the right-hand side of the second-last row of the tree and walks backwards through the array, calling *Max_Heapify* on each node.

Clearly, *Build_Max_Heap* builds a max heap and runs in $O(nlog(n))$ or better. In fact, since most nodes are near the bottom of the tree, they don't have far to go to float down, so that *Build_Max_Heap* runs in $O(n)$ time.

Heap Sort

To sort an array $A = a_0, ..., a_n$:

- Run *Build_Max_Heap*
- Move the tree root element to the end of the array. Consider the root as being removed from the heap, so that $a_0, ..., a_{n-1}$ represent the heap.
- Move a_{n-1} to the root position in the heap and call *Max_Heapify*
- Continue doing this until the heap is emptied

This means one call to *Build_Max_Heap* and n calls to *Max_Heapify*, which means $O(n + nlog(n))$ i.e. $O(nlog(n))$.

Priority Queues

A max priority queue should support the following operations: *Max*, *Extract_Max*, *Increase_Key*, *Insert*, *Decrease_Key*. We will leave *Decrease_Key* for later.

If we implement the queue as a max heap, then we can implement the priority-queue operations as follows:

Max (O(1))

Easy - just return the key of the root. $O(1)$.

Extract_Max (O(log n))

- Set m equal to the key at the root
- Swap the keys of the root and the last node
- Run *Max_Heapify* on the new root node
- Return m

Increase_Key (O(log n))

- Set the new key of the node
- While the node key is larger than its parent's key, swap the node and its parent

Insert (O(log n))

- Insert the desired node at the end of the heap with key $-\infty$ (so that the heap property is clearly preserved)
- Run *Increase_Key* to set the node to its desired key

Quicksort

Partition

The *Partition* operation takes an array A, rearranges it, and returns an index q such that $A[q]$ is $\geq$ everything that comes before it and $\leq$ everything that comes after it. How it works:

- Select the final element of the array as the *pivot*
- Imagine a contiguous selection (let us call it a *window*) of elements of the array, starting at the left and empty at first, containing elements $\geq$ *pivot*. As you move left to right through the array, the window will always be immediately to the left of the element which you are considering.
- Moving left to right through the array (but stopping before the final element), if you hit an element x which is $\leq$ *pivot*, then swap x with the leftmost element of the window - the effect will be to shunt the whole window over x (and to move the leftmost element of the window to be the rightmost element of the window).
- When you reach the final element (pivot) of the array, swap it with the leftmost element of the window.

Quicksort

```
def quicksort(A):
    pivot = Partition(A)
    quicksort(A[0, pivot - 1])
    quicksort(A[pivot + 1,])  # Missing final index means go to the end of the array
```

Notice that there is no merge step needed, because the *Partition* procedure ensures that, as long as the two subarrays are sorted, then the whole array is sorted.

Complexity

Space

Sorts in place, so has space complexity $O(1)$.

Time

Worst Case Notice that each *Partition* call is linear in the size of the subarray that it sorts. If the array is already sorted, then the pivot always stays at the end of the array and you end up making n calls to *Partition*. The first call costs n, the second costs $n-1$, ... , so the total cost is $\sum_{i=1}^{n} i = n(n+1)/2$, which is $O(n^2)$.

Average Case Won't treat this formally but will take two special cases.

Alternating Imagine the splits in a recursion tree (which would be $log_2(n)$ deep if every split were even). Since partitioning costs $O(n)$ at each level of the tree, we just need to compute the depth of the tree. Imagine that splits (i.e. the position of the pivot) alternate between perfect (pivot in the middle) and worst (pivot at the end), where all splits at one tree level are perfect, then they are all worst at the next level, and so on. Then the recursion tree has depth at most $2log_2(n)$, so the running time is still $O(nlog(n))$.

All Splits are 100:1 (or any constant ratio) Similarly to the "alternating" case, we just need to consider the depth of the recursion tree. This will be $log_{100}(n)$, so we still have a running time of $O(nlog(n))$.

Breadth-First Search

The Algorithm

We use vertex colours to indicate discovery state:

- White for undiscovered
- Grey for discovered but not finished

- Black for finished

```
def bfs(graph, source):
    for v in graph - {source}:
        vertex.colour = white
        vertex.dist = infty
        vertex.prev = None
    source.colour = grey
    source.dist = 0
    source.prev = NIL

    queue = [source]
    while not queue.empty():
        v = queue.pop()
        for nbr in v.neighbours():
            if nbr.colour == white:
                nbr.colour = grey
                nbr.dist = v.dist + 1
                nbr.prev = v
                queue.push(nbr)
        v.colour = black
```

Theorem: *BFS* finds all reachable vertices and assigns them *.distance* values which are equal to their shortest-path distances from the source vertex. **Proof:** Easy to see by induction on the shortest-path distance.

Complexity

No vertex is ever whitened, so each vertex is enqueued at most once and hence dequed at most once. Since a vertex's incident edges are scanned only when the vertex is dequeued, each edge is traversed at most once, so the complexity if $O(E + V)$.

Some Other Noteable Properties Of BFS

Lemma: At any point during *BFS*, the queue contains vertices whose *.distance* vales are non-strictly increasing from the queue's head to it's tail; morevoer the tail vertex's *.distance* value is at most one larger than the head vertex's. **Proof:** Not that easy to prove formally but think of it like this: if it were not the case, then it would violate the breadth-first nature of breadth-first search.

Immediately we get:

Corollary: Suppose that vertices a and b are enqueued during *BFS* and that a is enqueued before b. Then, when b is enqueued, we have $a.distance \leq b.distance$.

Depth-First Search

Let G be a directed or undierected graph.

```
def DFS(G):
    for each vertex v:
        v.colour = WHITE
        v.prev = NIL
    time = 0
    for each vertex v:
        if v.colour == WHITE
            DFS-VISIT(G, v, time)


def DFS-VISIT(G, v, time):
    ++time
    v.discovered = time
    v.color = GREY
    for each out-nbr n of v:
        if n.color == WHITE:
            n.prev = v
            DFS-VISIT(G, n, time)
    v.color = black
    ++time
    v.finished = time
```

Complexity: $O(V + E)$

A DFS run produces a depth-first forest (DFF) of depth-first trees (DFT), which have the obvious definitions.

Edge Classifications

- **Tree Edge:** An edge in the graph which belongs to a tree in the DFF
- **Forward Edge:** A non-tree edge connecting a vertex to one of its descendants in its depth-first tree
- **Back Edge:** An edge (obviously non-tree edge) connecting a vertex to one of its predecessors in its DFT
- **Cross Edge:** Any other edge (can go between vertices in the same DFT, as long as one vertex is not an ancestor of the other, or can go between vertices in different depth-first trees)

Theorems on Depth-First Search

The *Parenthesis Theorem* and *White Path Theorem* are very easy to see if you just think about what DFS does. Indeed, formal proofs almost obscure how simple these theorems are.

Theorem (Parenthesis Theorem): For any two vertices, exactly one of the following is true:

- the vertices' discover-finish time intervals are disjoint and neither vertex is a descendant of the other in the DFF
- one of the vertices' (let's say v's) discover-finish time interval is contained in the other's (let's say u's), and v is a descendant of u in a DFT

Theorem (White Path Theorem): In the DFF, a vertex v is descended from a vertex u if and only if, at the time that u is discovered, there is a path $u \to v$ consisting entirely of white vertices.

Corollary: A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof: Clearly, if there's a back edge, then there's a cycle. Suppose that G contains a cycle c. We show that a depth-first search of G yields a back edge. Let v be the first vertex to be discovered in c, and let (u, v) be the preceding edge in c. At time v is discovered, the vertices of c form a path of white vertices from v to u. By the white-path theorem, vertex u becomes a descendant of v in the DFF, so that (u, v) is a back edge.

Theorem: (Formulated by me, possibly suspect) For any collection S of vertices in an *undirected* graph, S is the set of vertices of a DFT if and only if it is the set of vertices of a connected component

Theorem: In a DFS of an *undirected* graph G, there are no cross edges.

Proof: This is officially a theorem. If my theorem above is correct, then it probably immediately gives the current theorem.

Topological Sort

Definition: A *topological sort* of a *directed* graph G is a sorting of the vertices of G such that every edge goes left-to-right in the vertex sorting.

Theorem: We can topologically sort a DAG by running DFS and sorting vertices in descending order of finishing time. **Proof:** Take any edge (u, v); we must show that v is finished before u. Consider the colour of v at the moment when the edge (u, v) is explored:

- Grey: Impossible because if v was grey, then u would be descended from v and there would be a cycle.
- White: v will become a descendant of u and hence will be finished before u.
- Black: v has already been finished and u has not (since we are exploring the edge (u, v)), so that v is finished before u.

Strongly Connected Components

The algorithm:

1. Call DFS on G and record finishing times of all vertices.
2. Call DFS on G^T (transpose of G), but in the main loop of DFS, consider the vertices in order of decreasing finishing times computed in step 1.
3. The SCCs are the vertices of the DFTs from step 2.

Definition: The *component graph* of a directed graph is the graph you get by identifying vertices which belong to the same SCC.

For any directed graph, the component graph is obviously a DAG.

We shall see that by considering vertices in the second DFS in decreasing order of the finishing times that were computed in the first DFS we are, in essence, visiting the vertices of the component graph in topologically sorted order.

When we refer to *v.discovered* and *v.finished* here, we shall always be referring to the first DFS run (the run on G as opposed to on its transpose graph).

For a set U of vertices, $d(U)$ shall refer to the earliest discovery time of any vertex in U; similarly, $f(U)$ shall refer to the latest finishing time of any vertex in U.

Lemma: If an edge crosses from SCC C to SCC C', then $f(C) > f(C')$. **Proof:** We split the proof into two cases:

- $d(C) < d(C')$: Let x be the first vertex discovered in C; then at time $x.d$, all vertices in C and C' are white and reachable from x, so that they all become descendants of x. It follows immediately that $f(C) > f(C')$.
- $d(C') < d(C)$: Let y be the first vertex discovered in C'. Then all vertices in C' become descendants of y but no vertices in C do (because, since there is an edge $C \to C'$, there can be no edge $C' \to C$). Hence $f(C') < d(C)$ and certainly $f(C') < f(C)$.

As a DAG, the component graph must have a topological ordering. In fact, we get a natural topological ordering as a corollary of the above lemma:

Corollary: If you line up the SCCs in decreasing order of finishing times, you get a topological ordering.

Rough Explanation Of Why The SCC Algorithm Works: Say you line up the SCCs in decreasing order of finishing times:

$$C_n, C_{n-1}, ..., C_0$$

Then by the above corollary, the above lineup is a reverse topological ordering in the G^T component graph, so that all edges go left. Now, the DFS on G^T - since it starts at the last finished vertex, starts at a vertex in C_n; clearly the first DFT hits all elements of C_n and, since all edges in the component graph go left, hits none of $C_{n-1}, ..., C_0$, so that indeed the first DFT hits exactly the

vertices of C_n. The second DFT similarly hits everything in C_{n-1} and similarly hits none of $C_{n-2}, ..., C_0$; it also hits nothing in C_n, since these vertices were already hit in the first DFT. Reasoning inductively like this, we see that the SCC algorithm does indeed identify the SCCs.

Minimum Spanning Trees (MSTs)

Definition: A *minimum spanning tree* for a connected acyclic graph G is a subtree where the sum of the edge weights is as small as possible.

Not that to specify a subtree of a graph, it is enough to specify the edges.

Generic Method

The generic method grows the MST one edge at a time, maintaining at each iteration a set of edges A which is a subset of some MST.

At each step, we determine an edge e that we can add to A without violating the invariant that A is a subset of some MST. We call such an edge a *safe edge* for A.

```
def Generic_Build_MST(graph):
    A = {}
    while A is not an MST:
        find an edge e that is safe for A
        add e to A
    return A
```

Recognising Safe Edges

Definition: A *cut* of a graph is just a partition of its vertices into two portions.

Definition: An edge of a graph *crosses* a cut if the edge's source and target are in different portions of the cut.

Definition: A cut of a graph *respects* a set A of edges if no edge of A crosses the cut.

Definition: A *light edge* satisfying a property P is an edge which is as light as possible among all edges satisfying P.

Theorem:

- Let A be a subset of edges that is included in some minimum spanning tree for G
- Let C be any cut of G that respects A

Then any light edge crossing C is safe for A.

Proof:

Let T be a MST that includes A and let (u, v) be a light edge crossing A. Aassume that T does not contain (u, v) (since we are done if it does). We will construct another MST T' which includes (u, v).

Consider the simple path p in T from u to v; we get a cycle by adding (u, v) to this path. Notice that, since u and v are on opposite sides of the cut C, at least one edge (x, y) in p also crosses C.

Note: (x, y) is not in A since A respects C.

Removing (x, y) from T will break T into two components; adding (u, v) back in will form a new spanning tree T'.

T' is clearly minimum since (u, v) is light.

Since $A \subseteq T$, we also have $A \cup (u, v) \subseteq T \cup (u, v)$. Since, as noted above, $(x, y) \notin A$, we have $A \cup (u, v) - (x, y) \subseteq T \cup (u, v) - (x, y) = T'$, so that (u, v) is safe for A.

Corollary: Let A be a set of edges that is contained in some MST. Then any light edge between connected components (i.e. trees) in the forest defined by A is safe for A.

Proof: Follows immediately from the above theorem and the fact that, for any connected component (tree) T, the cut $(V, V - T)$ respects A.

Kruskal's ALgorithm For Finding Safe Edges

So, the above theorem lets us recognise safe edges. The question now is: how do we find such safe edges? There are at least two algorithms for this, namely **Kruskal's Algorithm** and **Prim's Algorithm**. Here, we will treat Kruskal's algorithm only.

The idea of Kruskal's algorithm is as follows: we start by letting $A = \emptyset$ and creating a set T of trees by making each vertex its own tree. Then, considering edges of the graph in weakly increasing order of weight, we check if an edge (u, v) has both its edges in the same tree in T; if not, we unite the trees of u and v (reducing the size of T by 1) and add (u, v) to A.

```
def Kruskal(G,E):
    A = {}
    T = V  # consider each vertex as a tree
    increasing_edges = sort E in weakly increasing order of weight
    for (u,v) in increasing_edges:
        if tree(u) != tree(v):
            A.insert((u, v))
            unite(tree(u), tree(v))
    return A
```

Runtime Complexity Analysis: Need to know about disjoint-set data structures. Might come back to this. But the runtime complexity is $O(ElogE)$.

Noting that $|E| < |V|^2$, we get $log|E| = O(logV)$, so that we may restate the complexity as $O(ElogV)$.

Dijkstra Shortest Paths

In a weighted directed graph, let $\delta(u, v)$ represent the shortest weight of any path from u to v.

Terminiology: Suppose that s is our starting vertex and that v is another vertex. We denote by $v.d$ our current estimate of $\delta(s, v)$ and by $v.prev$ the predecessor of v in our current representative path $s \rightsquigarrow v$ of length $v.d$.

To begin the Dijkstra algorithm, we always call *InitialiseSingleSource*:

```
def InitialiseSingleSource(s):
    for v in G:
        v.d = infinity
        v.prev = NIL
    s.d = 0
```

The Dijkstra algorithm performs the *Relax* procedure exactly once on each edge:

```
def Relax(u, v):
    if v.d > u.d + w(u,v):
        v.prev = u
        v.d = u.d + w(u,v)
```

Lemma (Convergence Lemma): If $s \to ... \to u \to v$ is a shortest path and $u.d = \delta(s, u)$ at any time before (u, v) is relaxed, then $v.d = \delta(s, v)$ at all times afterwards. **Proof:** Obvious.

By the Convergence Lemma and induction on the length of the path, we easily get:

Lemma: If $v_0, v_1, ..., v_n$ is a shortest path from v_0 to v_n and the edges of this path are relaxed in order (namely in the order $(v_0, v_1), ..., (v_{n-1}, v_n)$), possibly with other relaxations in between, then $v_n.d = \delta(v_0, v_n)$ at all times after these relaxations.

Dijkstra's algorithm maintains a min-priority queue Q of vertices, keyed by their $.d$ values. The algorithm repeatedly extracts the min element of Q, and relaxes all edges leaving u.

```
def Init_Dijkstra(s):
    s.d = 0
    s.prev = NIL
    for v in G - {s}:
        v.d = infty
        v.prev = NIL
    Q = {}
```

```
    for v in G:
        Q.enqueue(v)

def Dijkstra(s):
    Init_Dijkstra(s)
    while !Q.empty()
        u = Q.extract_min()
        for each edge u->v:
            Relax(u,v)
```

Lemma (Upper Bound Property): At any time during a Dijkstra run, all vertices satisfy $\delta(s, v) \leq v.d$. **Proof:** Notice that the $.d$ values are always either ∞ or equal to the length of some path $s \to ... \to v$. The result follows immediately.

Theorem (Correctness Of Dijkstra): The Dijkstra algorithm terminates with $v.d = \delta(s, v)$ for all vertices v. **Proof:** We claim that, whenever a vertex v is dequeued, we have $v.d = \delta(s, v)$. Suppose not, and let v be the first vertex which fails this property; clearly $v.d > \delta(s, v)$ and $v \neq s$. Choose any shortest path $s \to ... \to v$, let q be the first vertex on this path which was still in Q immediately before v was dequeued, and let x be the predecessor of q on the shortest path. So we have $s \to ... \to x \to q \to ... \to v$ (where perhaps $s = x$ and $q = v$). Now

- $v.d \leq q.d$ because $v.d$ was minimal among vertices in Q
- $q.d = \delta(s, q)$ because
  - $x.d$ was equal to $\delta(s, x)$ immediately after x was dequeued (since v is the first vertex in the path which fails this property)
  - $x \to q$ was relaxed during the loop iteration where x was dequeued
- $\delta(s, q) \leq \delta(s, v)$ because q is before v in the shortest path

Putting the above equalities/inequalities together, we get $v.d \leq \delta(s, v)$, providing our contradiction.